

UNIVERSIDAD AUTÓNOMA DE BENI “JOSÉ BALLIVIAN”

Facultad de Ingeniería y Tecnología



Contenedores y orquestación: Docker y Kubernetes

Estudiantes:

1. JOEL LEIVA CALLE
2. JOSÉ DAVID CACEREZ RIVERO
3. JOSUÉ DAVID COLQUE CANDIA

Carrera: Ingeniería en sistemas

Docente: ING. HERMES RODRIGUEZ RIVERO

Materia: TECNOLOGÍAS EMERGENTES

Gestión: 2026

Contenido

INTRODUCCION	1
CONCEPTO DE CONTENEDOR	2
DOCKER: CONTENERIZACIÓN DE APLICACIONES	2
COMPONENTES BÁSICOS DE DOCKER	3
• Dockerfile:	3
• Imágenes de Docker:	3
• Contenedores:.....	3
INTRODUCCION A LA ORQUESTACION	3
BENEFICIOS DE LA ORQUESTACIÓN.....	4
KUBERNETES: ORQUESTADOR DE CONTENEDORES	5
COMPONENTES PRINCIPALES DE KUBERNETES	5
• Clúster:.....	5
• Nodo maestro (control plane):	5
• Nodo worker	5
• Pod:	5
• Servicios y despliegues:	5
RELACIÓN ENTRE DOCKER Y KUBERNETES	6
VENTAJAS Y DESVENTAJAS DE DOCKER	7
VENTAJAS Y DESVENTAJAS DE KUBERNETES	7
CONTENEDORES VS MAQUINAS VIRTUALES.....	8
DOCKER HUB.....	10
¿CÓMO USAR DOCKER?	11
¿CÓMO USAR KUBERNETES?	13
REFERENCIAS BIBLIOGRÁFICAS	14

INTRODUCCION

En la actualidad, la forma en que se desarrollan, despliegan y gestionan las aplicaciones ha cambiado significativamente gracias a las tecnologías de contenedores y orquestación, las cuales han transformado la manera en que las empresas construyen y administran sus sistemas de software.

Herramientas como Docker y Kubernetes se han convertido en componentes fundamentales dentro de la entrega moderna de aplicaciones, ya que permiten empaquetar software junto con todas sus dependencias, asegurando que funcione de manera consistente en distintos entornos, ya sea en desarrollo, pruebas o producción. Esto reduce problemas de compatibilidad y mejora la eficiencia en el despliegue de aplicaciones.

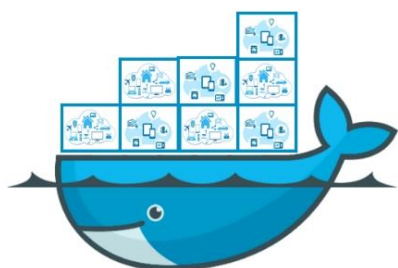
Docker facilita la creación de contenedores ligeros y portables a partir de imágenes, lo que permite ejecutar aplicaciones de forma aislada y reproducible. Por otro lado, Kubernetes se encarga de la orquestación de estos contenedores, gestionando su escalabilidad, disponibilidad y distribución automática en múltiples servidores.

Este informe explica el concepto de contenedor, el rol de Docker en la creación de imágenes y contenedores, y cómo Kubernetes se encarga de orquestar y administrar estos contenedores a gran escala, garantizando eficiencia, estabilidad y alta disponibilidad en los sistemas moderno

CONCEPTO DE CONTENEDOR

Un contenedor es una unidad ligera de software que empaqueta una aplicación junto con todas sus dependencias, configuraciones y bibliotecas necesarias para ejecutarla.

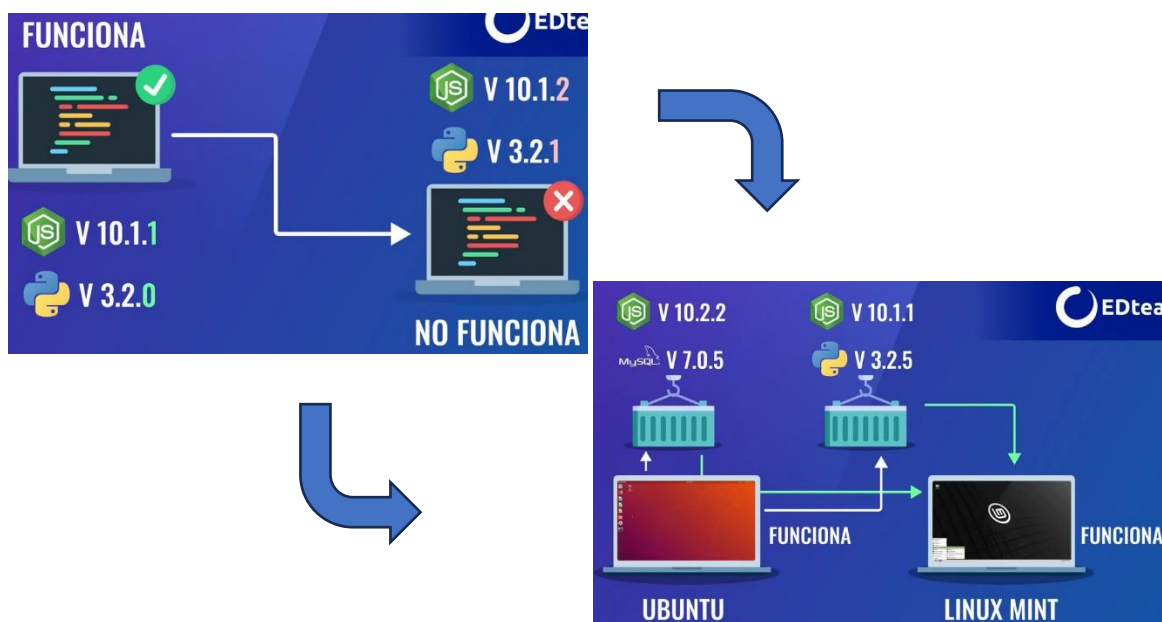
A diferencia de una máquina virtual completa, que incluye un sistema operativo invitado, el contenedor comparte el kernel del sistema host, lo que reduce su huella de recursos y permite un arranque más rápido.



DOCKER: CONTENERIZACIÓN DE APLICACIONES

Docker es una plataforma de código abierto que permite crear, empaquetar, distribuir y ejecutar aplicaciones en contenedores.

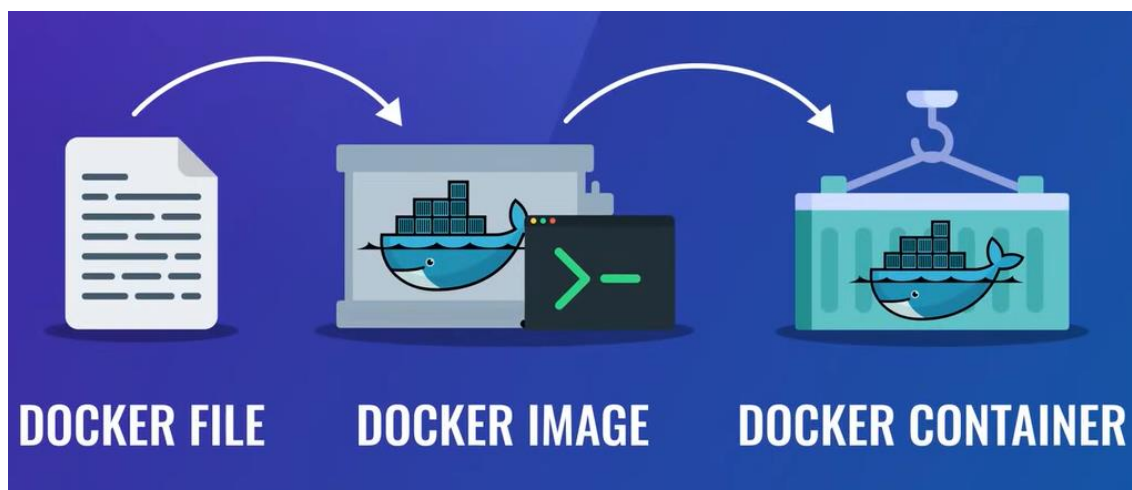
Su principal ventaja es que una aplicación construida con Docker puede ejecutarse de forma idéntica en cualquier sistema que tenga el runtime de contenedores instalado, reduciendo el problema clásico de “funciona en mi máquina”.



COMPONENTES BÁSICOS DE DOCKER

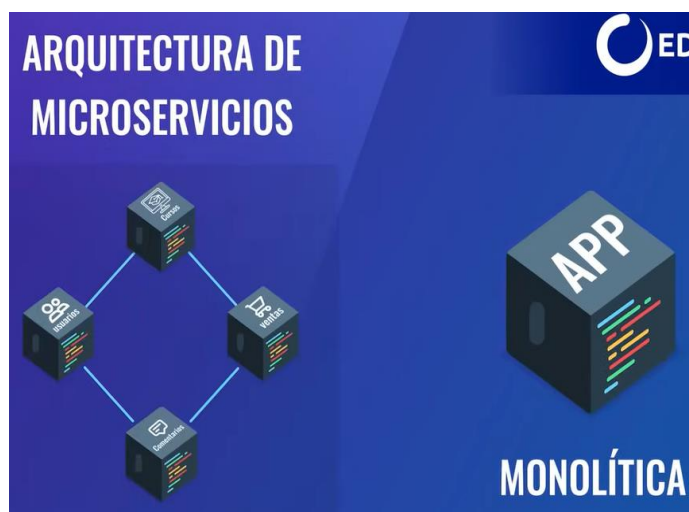
- **Dockerfile:** archivo de texto que contiene una secuencia de instrucciones para construir una imagen paso a paso (por ejemplo, instalar dependencias y copiar el código fuente).
- **Imágenes de Docker:** plantillas de solo lectura que definen el estado inicial de un contenedor; incluyen el sistema de archivos y las instrucciones para ejecutar la aplicación.
- **Contenedores:** instancias ejecutables de una imagen; pueden crearse, iniciarse, detenerse y eliminarse de forma rápida.

Docker automatiza la construcción de imágenes y la ejecución de contenedores, facilitando la integración continua, los despliegues reproducibles y la colaboración entre equipos de desarrollo y operaciones



INTRODUCCION A LA ORQUESTACION

Una aplicación puede construirse como un sistema monolítico (todo en un solo bloque) o mediante una arquitectura de microservicios, donde la aplicación se divide en múltiples partes independientes.



Debemos entender que una aplicación moderna no es una sola pieza, sino un conjunto de servicios que trabajan juntos; por ejemplo, una aplicación de comercio electrónico puede tener un servicio para usuarios, otro para productos, otro para pagos y otro para pedidos. Cada uno de estos servicios puede ejecutarse en su propio contenedor, lo que facilita su desarrollo, mantenimiento y escalabilidad. Sin embargo, cuando se tienen muchos contenedores funcionando al mismo tiempo, surge la necesidad de gestionarlos de manera eficiente. Aquí es donde entra la orquestación de contenedores, utilizando herramientas como Kubernetes, que permite organizar, coordinar y automatizar el funcionamiento de todos estos servicios, asegurando que la aplicación funcione correctamente como un todo.

La **orquestación de contenedores** es el proceso automatizado de despliegue, escalado, monitoreo y recuperación de contenedores en un entorno distribuido.



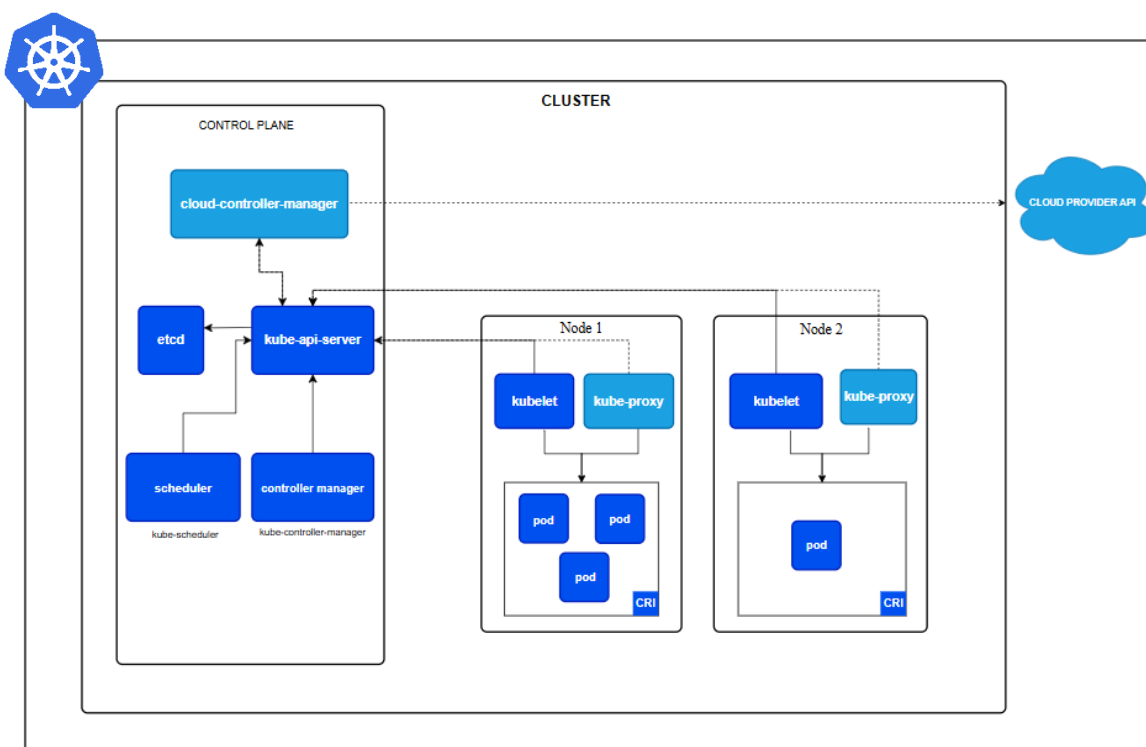
BENEFICIOS DE LA ORQUESTACIÓN

- Reducción de la complejidad operacional al automatizar tareas repetitivas.
- Mejora de la disponibilidad y resiliencia, ya que el orquestador puede reiniciar contenedores que fallen y redistribuir la carga.
- Optimización de recursos y reducción de costos, al aprovechar mejor la infraestructura y evitar aprovisionamiento excesivo

KUBERNETES: ORQUESTADOR DE CONTENEDORES

Kubernetes (también conocido como K8s) es una plataforma de código abierto de orquestación de contenedores creada por Google y mantenida por la Cloud Native Computing Foundation (CNCF). Kubernetes permite gestionar grupos de contenedores (normalmente Docker) en un clúster de nodos, automatizando el despliegue, el escalado y la recuperación de aplicaciones.

COMPONENTES PRINCIPALES DE KUBERNETES



- **Clúster:** conjunto de nodos (servidores físicos o virtuales) donde se ejecutan los contenedores.
- **Nodo maestro (control plane):** responsable de gestionar el estado del clúster, aceptar peticiones de usuario y programar contenedores en los nodos.
- **Nodo worker:** nodo donde se ejecutan realmente los contenedores de la aplicación.
- **Pod:** unidad mínima de despliegue; puede contener uno o más contenedores que comparten recursos de red y almacenamiento.
- **Servicios y despliegues:** abstraen cómo se accede a los pods y cómo se escalan o actualizan sin interrumpir el servicio.

Con Kubernetes es posible desplegar aplicaciones compuestas por cientos de microservicios, escalarlos según la carga de trabajo y recuperarlos automáticamente ante fallos.

RELACIÓN ENTRE DOCKER Y KUBERNETES



Docker y Kubernetes no son tecnologías rivales, sino complementarias. Docker se enfoca en crear y empaquetar aplicaciones en contenedores, mientras que Kubernetes se encarga de orquestar y gestionar esos contenedores en un entorno de producción.



En la práctica, un flujo típico es:

1. Crear un Dockerfile y construir una imagen de Docker.
2. Empujar la imagen a un registro de contenedores (por ejemplo, Docker Hub o un registro privado).
3. Desplegar y gestionar contenedores basados en esa imagen mediante Kubernetes en un clúster.

Esta combinación permite entornos de desarrollo, pruebas y producción muy similares, lo que facilita la automatización y la integración continua (CI) y entrega continua (CD).

VENTAJAS Y DESVENTAJAS DE DOCKER

VENTAJAS	DESVENTAJAS
Portabilidad: se ejecuta en cualquier sistema o nube sin cambios.	Seguridad: requiere buena configuración para evitar vulnerabilidades.
Entornos reproducibles: funciona igual en desarrollo y producción.	Persistencia de datos: se deben gestionar volúmenes correctamente.
Rapidez de despliegue: los contenedores inician en segundos.	Curva de aprendizaje: puede ser complejo al inicio.
Aislamiento: cada aplicación funciona de forma independiente.	Dependencia del sistema host: comparte el kernel del sistema operativo.
Uso eficiente de recursos: consume menos que máquinas virtuales.	Limitaciones en aplicaciones muy complejas o específicas.
Facilita microservicios: permite dividir aplicaciones en partes pequeñas.	Requiere gestión adicional en entornos grandes.

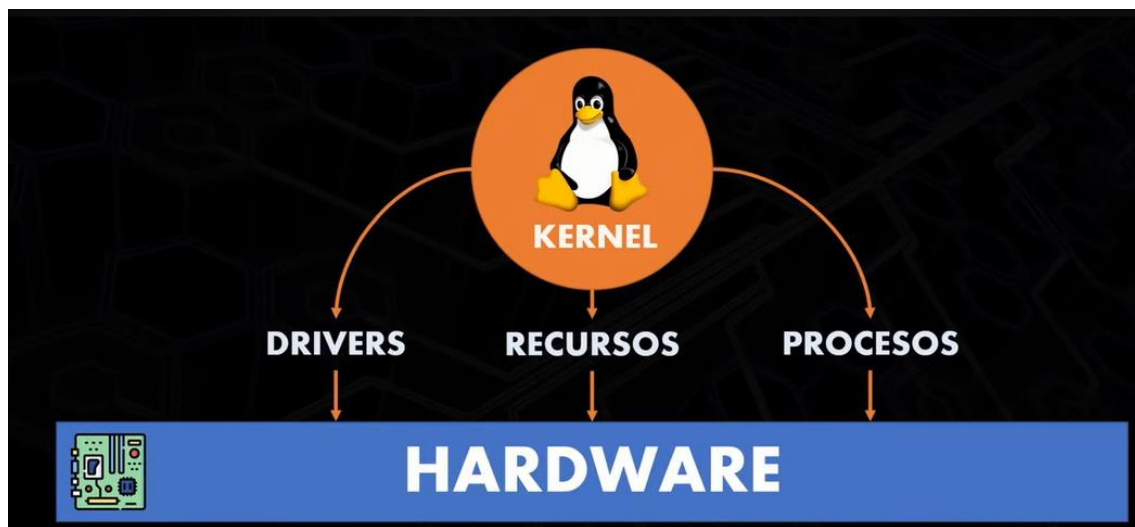
VENTAJAS Y DESVENTAJAS DE KUBERNETES

VENTAJAS	DESVENTAJAS
Escalabilidad automática: ajusta contenedores según la demanda.	Complejidad: difícil de aprender y configurar al inicio.
Alta disponibilidad: reemplaza contenedores fallidos automáticamente.	Sobrecarga: no es ideal para aplicaciones pequeñas.
Gestión centralizada: administra redes, almacenamiento y configuraciones.	Consumo de recursos: requiere más infraestructura.
Automatización: despliegue, actualización y mantenimiento automáticos.	Configuración compleja: requiere múltiples archivos y ajustes.
Balanceo de carga: distribuye el tráfico entre contenedores.	Mantenimiento: necesita monitoreo y administración constante.
Portabilidad en la nube: funciona en diferentes proveedores cloud.	Seguridad: requiere configuración avanzada de accesos y redes.

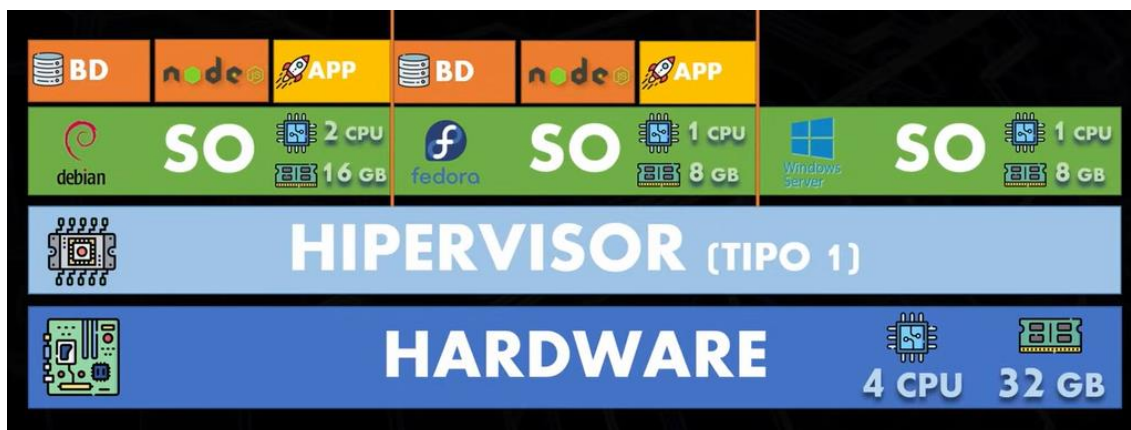
CONTENEDORES VS MAQUINAS VIRTUALES

kernel

Gracias al kernel del sistema operativo, los contenedores (como los de Docker) pueden funcionar de manera ligera y eficiente, ya que no necesitan un sistema operativo completo propio, sino que comparten el mismo núcleo, lo que permite un mejor uso de los recursos, un arranque más rápido y un adecuado aislamiento entre aplicaciones mediante mecanismos como namespaces y cgroups.



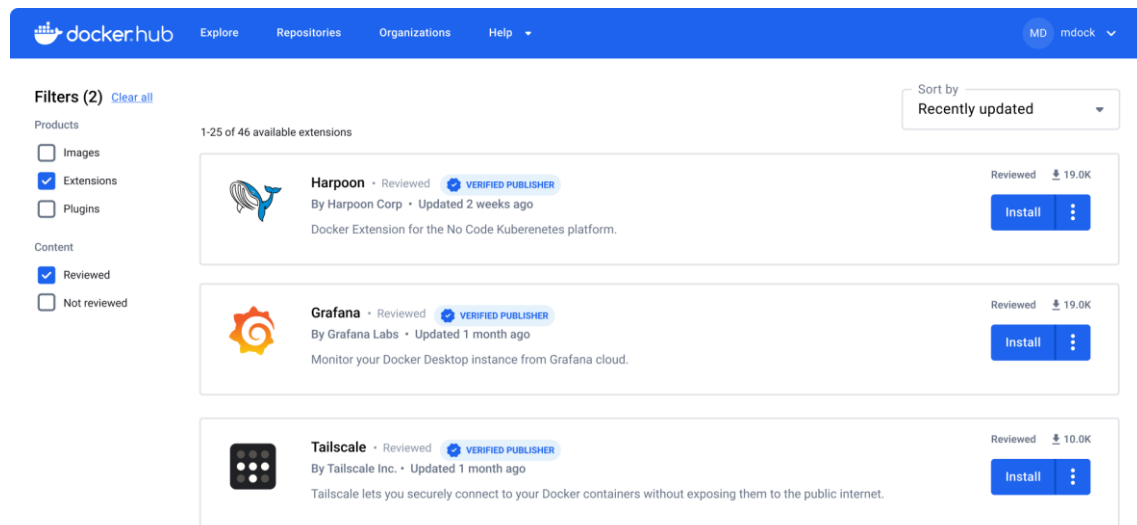
Mientras tanto, otra opción puede ser crear una máquina virtual, pero en este caso tendríamos que contar con un hipervisor para gestionar las máquinas virtuales e instalar un sistema operativo completo en cada una, lo que implica un mayor consumo de recursos, tiempos de arranque más lentos y una menor eficiencia en comparación con los contenedores, que comparten el kernel del sistema.



DIFERENCIAS

Aspecto	Contenedores (ej. Docker)	Máquinas Virtuales (VM)
Base de aislamiento	Procesos aislados sobre el mismo kernel.	Sistemas completos con su propio SO y kernel.
Uso de recursos (CPU/RAM)	Menor consumo; se comparte el kernel.	Mayor consumo; cada VM tiene su propio SO.
Tiempo de arranque	Segundos, muy rápido.	Más lento (iniciar SO completo).
Portabilidad	Alta; funciona igual en desarrollo, pruebas y producción.	Más rígida; depende de la configuración de la VM.
Aislamiento y seguridad	Aislamiento parcial; riesgo si el kernel se compromete.	Aislamiento más fuerte; cada SO es independiente.
Escalabilidad y densidad	Se pueden ejecutar muchos contenedores en el mismo host.	Menor densidad por uso de recursos.

DOCKER HUB



Docker Hub es un repositorio en la nube donde se almacenan, comparten y distribuyen imágenes de contenedores Docker, funcionando como una biblioteca centralizada de imágenes listas para usar. Allí los desarrolladores pueden encontrar imágenes oficiales de sistemas operativos, lenguajes y bases de datos (como Ubuntu, Nginx o MySQL), así como subir y compartir sus propias imágenes, lo que facilita la colaboración, la integración continua y el despliegue rápido de aplicaciones en cualquier entorno que use Docker.

¿CÓMO USAR DOCKER?

Docker se utiliza mediante comandos en la terminal para crear, ejecutar y gestionar contenedores. Su flujo básico sigue estos pasos:

1. Instalación

Primero se instala Docker desde la página oficial de Docker. Después de instalarlo, se verifica que esté funcionando:

```
docker --version
```

2. Descargar una imagen

Una imagen es la base de un contenedor. Se descarga desde Docker Hub:

```
docker pull nginx
```

Esto descarga una imagen de servidor web llamada Nginx.

3. Crear y ejecutar un contenedor

Con una imagen se puede crear un contenedor:

```
docker run -d -p 80:80 nginx
```

- run: crea y ejecuta el contenedor
- -d: en segundo plano
- -p: conecta puertos del sistema con el contenedor

4. Ver contenedores activos

```
docker ps
```

Muestra los contenedores que están en ejecución.

5. Detener un contenedor

```
docker stop ID
```

Detiene el contenedor en ejecución.

6. Eliminar un contenedor

```
docker rm ID
```

Elimina el contenedor que ya no se usa.

7. Ver imágenes disponibles

```
docker images
```

Muestra todas las imágenes descargadas en el sistema.

8. Crear tu propia imagen (opcional)

Se usa un archivo llamado **Dockerfile**:

```
FROM python:3
```

```
WORKDIR /app
```

```
COPY . .
```

```
CMD ["python", "app.py"]
```

Luego se construye la imagen:

```
docker build -t mi_app .
```

Y se ejecuta:

```
docker run mi_app
```

Docker funciona en tres pasos principales:

1. Descargar una imagen
2. Crear un contenedor
3. Ejecutarlo y gestionarlo

¿CÓMO USAR KUBERNETES?

1. Tener Kubernetes instalado

Normalmente se usa con herramientas como:

- Minikube (local)
- Docker Desktop con Kubernetes
- Clusters en la nube (Google Cloud, AWS, Azure)

2. Verificar el clúster

```
kubectl cluster-info
```

3. Ver nodos del clúster

```
kubectl get nodes
```

4. Crear un pod (contenedor básico)

```
kubectl run mi-pod --image=nginx
```

5. Ver pods en ejecución

```
kubectl get pods
```

6. Exponer una aplicación (crear servicio)

```
kubectl expose pod mi-pod --type=NodePort --port=80
```

7. Ver servicios

```
kubectl get services
```

8. Eliminar un pod

```
kubectl delete pod mi-pod
```

Kubernetes usa el comando **kubectl** para administrar contenedores (Docker normalmente).

Kubernetes funciona así:

1. Recibe contenedores (generalmente de Docker)
2. Los organiza en Pods
3. Los ejecuta dentro de un clúster
4. Los escala y mantiene automáticamente

REFERENCIAS BIBLIOGRÁFICAS

1. SentinelOne. “Kubernetes frente a Docker: explicación de las diferencias clave”. Disponible en: <https://www.sentinelone.com/es/cybersecurity-101/cloud-security/kubernetes-vs-docker/>
2. Stackscale. “Contenedores, sistemas de contenerización y orquestadores”. Disponible en: <https://www.stackscale.com/es/blog/contenerizacion-orquestacion-contenedor/>
3. LatinCloud. “Docker y Kubernetes: Guía Completa de Implementación y Beneficios”. Disponible en: <https://blog.latincloud.com/dockers-y-kubernetes-guia-implementacion/>
4. Universidad Carlos III de Madrid (UC3M). “Orquestación de contenedores con Kubernetes”. Disponible en: <https://e-archivo.uc3m.es/rest/api/core/bitstreams/ccb5e1ee-c88d-45e7-a594-3d02fc9d959f/content>
5. Palentino. “De cero a cien con Docker y Kubernetes”. Disponible en: <https://www.palentino.es/pdfs/De-cero-a-cien-con-Docker-y-Kubernetes.pdf>
6. IBM. “¿Qué es la orquestación de contenedores?”. Disponible en: <https://www.ibm.com/mx-es/think/topics/container-orchestration>
7. PlusTecnología. “Docker y Kubernetes: Guía Práctica Completa 2025”. Disponible en: <https://plustecnologia.com/blog/docker-kubernetes-guia-practica-2025>
8. Oracle América Latina. “Comparación entre Kubernetes y Docker”. Disponible en: <https://www.oracle.com/latam/cloud/cloud-native/kubernetes-engine/what-is-kubernetes/kubernetes-vs-docker/>
9. Palo Alto Networks. “¿Qué es la orquestación de contenedores?”. Disponible en: <https://www.paloaltonetworks.lat/cyberpedia/what-is-container-orchestration>
10. Atlassian. “Comparación de contenedores y máquinas virtuales”. Disponible en: <https://www.atlassian.com/es/microservices/cloud-computing/containers-vs-vms>
11. Kaseya. “Contenedores frente a máquinas virtuales (VM): todo lo que necesitas saber”. Disponible en: <https://www.kaseya.com/es-la/blog/containers-vs-virtual-machines-vm/>
12. IONOS. “Docker vs. Virtual Machines: las diferencias principales”. Disponible en: <https://www.ionos.com/es-us/digitalguide/servidores/know-how/docker-vs-virtual-machines/>
13. IBM. “Contenedores vs. máquinas virtuales (VM): ¿cuál es la diferencia?”. Disponible en: <https://www.ibm.com/es-es/think/topics/containers-vs-vms>

14. IBM. “Kernel - El núcleo del sistema operativo”. Disponible en: <https://www.ionos.com/es-us/digitalguide/servidores/know-how/que-es-el-kernel/>
15. Microsoft Learn. “Acerca de los contenedores de Windows”. Disponible en: <https://learn.microsoft.com/es-es/virtualization/windowscontainers/about/>
16. Docker Inc. “Docker Hub Container Image Library”. Disponible en: <https://hub.docker.com>
17. KeepCoding. “Docker Hub: ¿Qué es y cómo funciona? Guía 2026”. Disponible en: <https://keepcoding.io/blog/que-es-docker-hub/>
18. Chakray. “¿Qué es Docker y para qué funciona? Explora ventajas y utilidades de esta potente plataforma”. Disponible en: <https://chakray.com/es/que-es-docker-que-funciona-explorando-las-ventajas-utilidades-esta-potente-plataforma/>